
ANTIBIOTIC RESISTANCE PREDICTION FROM GENOMIC K-MER FEATURES: A BLENDED XGBOOST AND LOGISTIC REGRESSION APPROACH

Yanis Kadri*

Matricule: 1946507
Polytechnique Montréal
yanis.kadri@polymtl.ca

Félix Delisle

Matricule: 2140727
Polytechnique Montréal
felix.deslisle@polymtl.ca

Iheb Ouni

Matricule: 2330087
Polytechnique Montréal
iheb.ouni@polymtl.ca

ABSTRACT

Our approach combines XGBoost and Logistic Regression in a blended ensemble, achieving a public leaderboard score of **0.81623** (Macro F1). Key findings include: (1) chi-squared feature selection significantly outperforms dimensionality reduction techniques like SVD, (2) metadata features introduce noise and hurt generalization, (3) StratifiedKFold validation provides better Kaggle score correlation than GroupKFold, and (4) conservative blend weights that favor the linear model improve generalization beyond the OOF-optimized weights.

*Team name: Overfitting Enjoyers.

1 INTRODUCTION

Antibiotics resistance is a global health challenge that machine learning can help solving. The goal of this competition was to develop an algorithm tasked with predicting whether given bacteria would be resistant or susceptible to antibiotics based on k-mer (short subsequences of DNA that capture local patterns) frequency features extracted from whole-genome sequences.

The dataset presents several challenges:

- **High dimensionality:** 1 000 000 k-mer features with only 1 939 training samples
- **Class imbalance:** (6.1:1 ratio)
- **Limited training data:** Risk of overfitting with complex models

This report will outline the steps we took while attempting to solve this problem.

2 FEATURE DESIGN

2.1 DATA OVERVIEW

The training data consists of pre-computed k-mer frequency vectors with 1 million features per sample. Metadata files were also provided.

2.2 PREPROCESSING

Given the nature of k-mer count features, we applied minimal preprocessing, for several reasons. First, tree-based models such as XGB are not impacted by *classical* normalizations (2) so aggressive normalization risks diminishing the representative power of some k-mers for no added value.

For Logistic Regression, `MaxAbsScaler` (scaling non-zero entries in the $[-1, 1]$ range) was used to preserve sparsity and memory efficiency. Unlike `StandardScaler`, which we used at first, but which densified the matrix and caused memory issues.

Our preprocessing pipeline consisted only of: (1) casting to `float32` for memory efficiency and because XGB uses it internally, (2) chi-squared feature selection to reduce dimensionality, and (3) `MaxAbsScaler`. This minimal approach outperformed more complex preprocessing strategies in our experiments.

2.3 FEATURE SELECTION STRATEGY

Given the extreme dimensionality ($p \gg n$), aggressive feature selection is essential (to avoid the curse of dimensionality and ensure computational feasibility on our devices). We evaluated two approaches:

Chi-squared Selection (SelectKBest): We use the chi-squared statistic to rank features by their association with the target. Because this statistic is computed only on non-zero entries and uses simple frequency statistics, this approach is computationally efficient and good for sparse and high-dimensional data. We selected $k = 50000$ features for XGBoost and $k = 70000$ for Logistic Regression. These values were determined empirically via structured exploration: we evaluated model performance on Kaggle while increasing and decreasing k , and retained the values that obtained the best public scores.

Truncated SVD: We also tested dimensionality reduction via Truncated SVD, testing for different projection sizes, ranged from 128 to 1024 components. However, this approach consistently underperformed `SelectKBest` on the Kaggle leaderboard (*approx* 0.78 vs 0.80), likely because SVD forms linear combinations that obscure the features, losing their representative power along the way.

Pruning the features while cross validating their optimal count was shown to be the best approach.

2.4 METADATA EXCLUSION

The competition provided metadata such as isolation source, location, and collection date. Our initial experiments excluded these variables. To assess whether we should include them, we ran adversarial validation using metadata only and the classifier performed close to random, indicating no strong shift.

However, we tested incorporating metadata using one-hot encoding and simple feature engineering (e.g., location clustering and source simplification). In all cases, models including metadata achieved lower Kaggle scores (0.797 vs. 0.816 for identical algorithms).

We hypothesize that metadata encourages the learning of noisy or contextual correlations that do not generalize well. Since k-mer features alone provided a strong and stable signal, metadata was excluded from the final model.

3 ALGORITHMS

3.1 XGBOOST

XGBoost(1) is a boosting algorithm well suited to high-dimensional, tabular and sparse data. Its built-in regularization, class imbalance handling, and histogram-based training (continuous features made into discrete groups, reducing computation) makes it effective in our setting.

3.2 LOGISTIC REGRESSION

Logistic Regression, as a linear model, provides a convenient baseline offering a different inductive bias than XGB, class weighting to handle imbalanced data (as in our case), and L2 regularization to control model complexity in high-dimensional contexts.

3.3 ENSEMBLE STRATEGY

We combined both models via probability averaging:

$$p_{blend} = w_{XGB} \cdot p_{XGB} + w_{LR} \cdot p_{LR} \quad (1)$$

The optimal weight w_{XGB} was determined through grid search over the validation predictions (OOB). This process is detailed in the following section.

4 METHODOLOGY

4.1 ALGORITHM SELECTION

The first model, we built was a model using a pipeline of `TruncatedSVD` and `Logistic Regression`. The results were quite unsatisfying but having a running model at this step was encouraging.

We then ran a Grid-Search with different models, several times with different hyperparameters (`LogReg`, `SVM`, `RandomForest`, `XGB`, `DecisionTree`) to get a sense of which one we should use. Besides XGB, all of them were overfitting (way higher validation than test F1), so we decided at first to stick with XGB.

After tuning XGB with `GridSearchCV` (see 4.4) we tried different blends, with as shown in Table 5. Further experiments showed that blending with a logistic regression improved the model's stability.

4.2 LEAKAGE PREVENTION

We employed several strategies to prevent leakage, because our initial runs did feature selection before CV, hence leaking information about the validation set to the best features and got highly-optimistic OOB scores.

To solve that, all preprocessing steps (feature selection, scaling) were encapsulated within `sklearn.pipeline.Pipeline` objects. This ensures that (`SelectKBest`) is fit only on training fold data, and (`MaxAbsScaler`) learns parameters only from training folds.

4.3 CROSS-VALIDATION STRATEGY

We compared two cross-validation strategies :

GroupKFold : Defined using isolation source and location (metadata) to enforce that samples from the same groups never appear in both training and validation sets, simulating a realistic scenario with unseen sources. This resulted in an out-of-fold F1 of 0.78 and a Kaggle score of 0.797.

StratifiedKFold : Aims to preserve class proportions across folds. This strategy achieved a higher out-of-fold F1 of 0.80 and a stronger Kaggle score of 0.816.

The better alignment of StratifiedKFold with Kaggle performance suggests that the test set follows a distribution similar to the training data.

4.4 HYPERPARAMETER SEARCH

We employed a two-stage hyperparameter optimization strategy on our initial XGB-only strategy. In the first stage, we performed grid search using 5-fold StratifiedKFold cross-validation with macro F1 as the scoring metric.

Parameter	Search Space
max_depth	{3, 4, 7}
learning_rate	{0.03, 0.05}
n_estimators	{400, 600}
subsample	{0.7, 0.8}
colsample_bytree	{0.7, 0.8}

Table 1: XGBoost grid search space.

We used the same approach for our Logistic Regression hyperparameter search. The final results can be found in Table 4.

In the second stage, we manually refined hyperparameters based on Kaggle leaderboard feedback. We discovered that :

- Increasing feature selection from $k = 10000$ to $k = 50000$ significantly improved performance. But going over it reduced it (see Table 2 and Table 3).
- Adding L1/L2 regularization (`reg_alpha=0.1`, `reg_lambda=1.0`) improved generalization
- `max_depth=5` provided the best bias-variance tradeoff

The final XGBoost configuration (see Table 4) used a mix of fixed hyperparameters and cross-validated selection, as the grid search optimum did not always transfer to the test set.

4.5 BLEND WEIGHT OPTIMIZATION

To determine the optimal blend proportions for our 2 algorithms, we used a cross-validation algorithm using the formula below:

$$w_{XGB}^* = \underset{w \in [0,1]}{\text{argmax}} F1_{macro}(y_{train}, \mathbf{1}[w \cdot p_{XGB}^{OOF} + (1 - w) \cdot p_{LR}^{OOF} \geq 0.5]) \quad (2)$$

We evaluated 21 (arbitrary) evenly-spaced w values from 0 to 1 (with a step size of 0.05) and with a fixed threshold of 0.5. The optimized weight was $w_{XGB}^* = 0.75$, achieving an OOF F1 of 0.81.

However, we observed that this w (75% XGB) achieved only 0.814 on the public leaderboard, while a more conservative weight (65% XGB) achieved **0.816**. This suggests the OOF-optimized weight slightly overfit to the training distribution, and giving more weight to the simpler linear model improved generalization.

5 RESULTS

This section consolidates the experimental evidence presented throughout the report, especially in the Methodology section, and provides a comparative analysis of the model choices, hyperparameters, and ensemble configurations. We chose to include some of the results detail in the previous sections to improve readability and overall flow of the report.

5.1 IMPACT OF KEY HYPERPARAMETERS

Among all tested hyperparameters, the number of selected k-mer features (k) had the largest impact on performance. Table 2 summarizes the effect of increasing k for both XGB and Logistic Regression.

Performance improved up to $k = 50000$, after which it reversed, suggesting overfitting. This trend was consistent across cross-validation and Kaggle submissions.

Another important hyperparameter was `scale_pos_weight` which tells XGBoost to weight misclassifications of the minority class (resistant) higher during training to compensate for class imbalance. This was computed as the ratio of negative to positive classified samples.

Furthermore, the blend proportions were also an important hyperparameter obtained through cross-validation.

5.2 MODEL COMPARISON

Table 3 and Figure 1 presents a consolidated comparison of the main modeling approaches explored during the competition. Single-model approaches based on Truncated SVD consistently underperformed feature selection via chi-squared statistics. Tree-based ensembles outperformed linear models, but also exhibited higher variance.

Among ensemble methods, blending XGBoost with Logistic Regression yielded better generalization than combining two tree-based models, suggesting that inductive bias diversity was more beneficial than raw model strength.

5.3 FINAL MODEL SELECTION

The final model was selected based on leaderboard performance, generalization behavior, and model simplicity. A blend favoring Logistic Regression (65% / 35%) achieved the highest public leaderboard score (0.816), outperforming more complex ensembles.

Figure 2 provides a visual summary of the relative performance of the different blends we have tried for those 2 models.

6 DISCUSSION

Having achieved third place *ex-aequo* on the public leaderboard, we were initially confident in the robustness of our approach. The minimal spread between OOF validation and public test scores suggested limited overfitting and good generalization. However, performance dropped substantially on the private test set, with the macro F1 decreasing from approximately 0.81 to 0.69. This discrepancy suggests we indeed were overfitting a lot. Here is what we can discuss about our approach :

6.1 STRENGTHS

The main strength of our approach lies in its simplicity and stability. The use of a minimal preprocessing pipeline allowed for reproducible results and facilitated hyperparameter exploration. Ensembling two models with complementary inductive biases proved particularly effective, as the Logistic Regression component acted as a regularizer and improved generalization. Additionally, explicitly handling class imbalance and avoiding metadata dependencies contributed to a more robust model.

6.2 LIMITATIONS

Several things may explain our performance drop on the private test set. Chi-squared feature selection independently evaluates features and ignores interactions between them, which may encode important biological relationships. We overlooked that. Hyperparameters were optimized in such a way that they may not be robust to subtle shifts between datasets. Moreover, repeated public leaderboard feedback likely introduced mild implicit overfitting, even when OOF metrics appeared stable.

6.3 FUTURE WORK

Future improvements could incorporate models that better exploit sequence structure, such as attention-based architectures operating directly on k-mer patterns. We could use better dimensionality reduction techniques. Finally, integrating biological knowledge, such as known resisting genes, could help guide feature selection and reduce the reliance on purely statistical signals.

7 STATEMENT OF CONTRIBUTIONS

Yanis Kadri: Implemented ensemble strategies, metadata experiments, cross-validation comparison. Wrote Methodology and Results sections.

Félix Delisle: Developed initial XGBoost pipeline, feature selection experiments, hyperparameter tuning. Wrote Feature Design and Algorithms sections.

Iheb Ouni: Created visualizations, wrote Discussion section and edited the full report.

We hereby state that all the work presented in this report is that of the authors.

Overleaf’s Writefull model as well as google’s Gemini Pro 3 model have helped us proofread the text in this report. Only grammar, syntax and phrasing have been edited with those tools.

REFERENCES

- [1] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785-794).
- [2] Pinheiro, J. M. H., de Oliveira, S. V. B., Silva, T. H. S., Saraiva, P. A. R., Souza, E. F., Godoy, R. V., Ambrosio, L. A., & Becker, M. (2025). The impact of feature scaling in machine learning: Effects on regression and classification tasks. *IEEE Access*, 13, 199903–199931.

A APPENDIX: FEATURE SELECTION HYPERPARAMETER SEARCH

Table 2: Feature selection k values tested		
k	XGB OOF F1	LR OOF F1
10 000	0.78	0.72
30 000	0.79	0.74
50 000	0.80	0.76
70 000	0.79	0.76
200 000	0.77	0.77

B APPENDIX: EXPERIMENTAL RESULTS SUMMARY

Table 3: Experimental Results Summary			
Model	Features	Public LB	Private LB
<i>Early Experiments (Single Models)</i>			
XGB + SMOTE + SVD(200)	200 components	0.754	0.694
XGB + SVD(500)	500 components	0.761	0.705
XGB + SelectKBest(10k)	10000 features	0.763	0.677
XGB + SelectKBest(50k)	50000 features	0.801	0.735
XGB + SelectKBest(200k)	200000 features	0.786	0.726
<i>Metadata Experiments</i>			
XGB + SVD(128) + Metadata	128 + meta	0.790	0.718
<i>Ensemble Experiments</i>			
XGB + CatBoost blend	50k (each)	0.809	0.728
XGB + LR blend (75/25, t=0.5)	50k + 70k	0.814	0.705
XGB + LR blend (80/20, t=0.5)	50k + 70k	0.811	0.705
<i>Final Model</i>			
XGB + LR blend (65/35, t=0.5)	50k + 70k	0.816	0.695

C APPENDIX : FINAL HYPERPARAMETERS

Table 4: Final hyperparameters for both models

Model	Parameter	Value
XGBoost	n_estimators	500
	max_depth	5
	learning_rate	0.05
	reg_alpha	0.1
	reg_lambda	1.0
	subsample	0.7
	colsample_bytree	0.7
	scale_pos_weight	6.10
Logistic Reg.	C	1.0
	penalty	L2
	class_weight	balanced
	solver	liblinear

D APPENDIX : MODEL PERFORMANCE COMPARISON (TABLE)

Table 5: Model comparison. Best Kaggle score in bold.

Model	Features	OOF F1	Kaggle
XGBoost Only	50k	0.800	0.800
Logistic Regression	70k	0.760	0.724
XGBoost + Metadata	50k + meta	0.797	0.797
XGB + CatBoost	50k	0.815	0.808
XGB + LR (75/25, OOF-optimized)	50k/70k	0.810	0.814
XGB + LR (65/35, conservative)	50k/70k	0.805	0.816

E APPENDIX : MODEL PERFORMANCE COMPARISON (GRAPH)

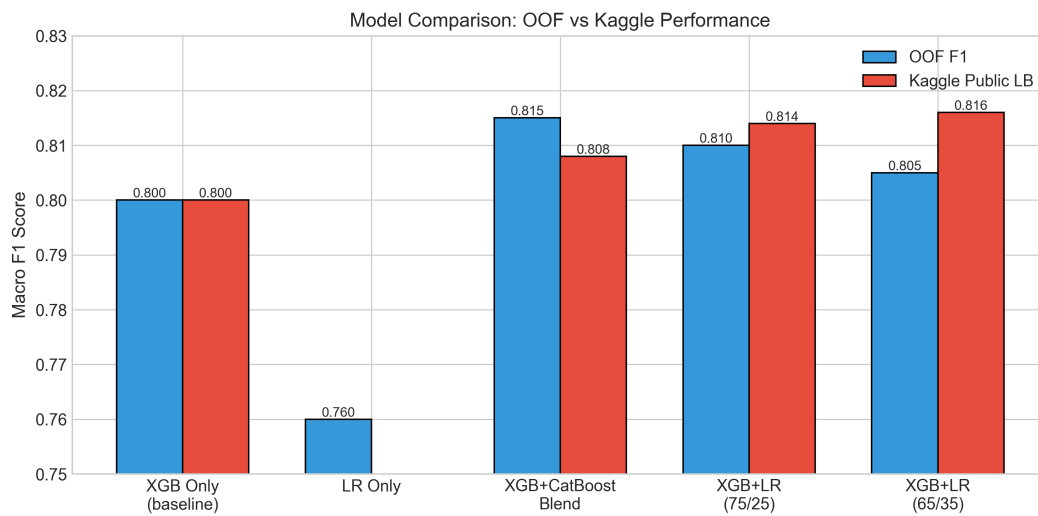


Figure 1: Model performance comparison

F APPENDIX : BLEND WEIGHT COMPARISON

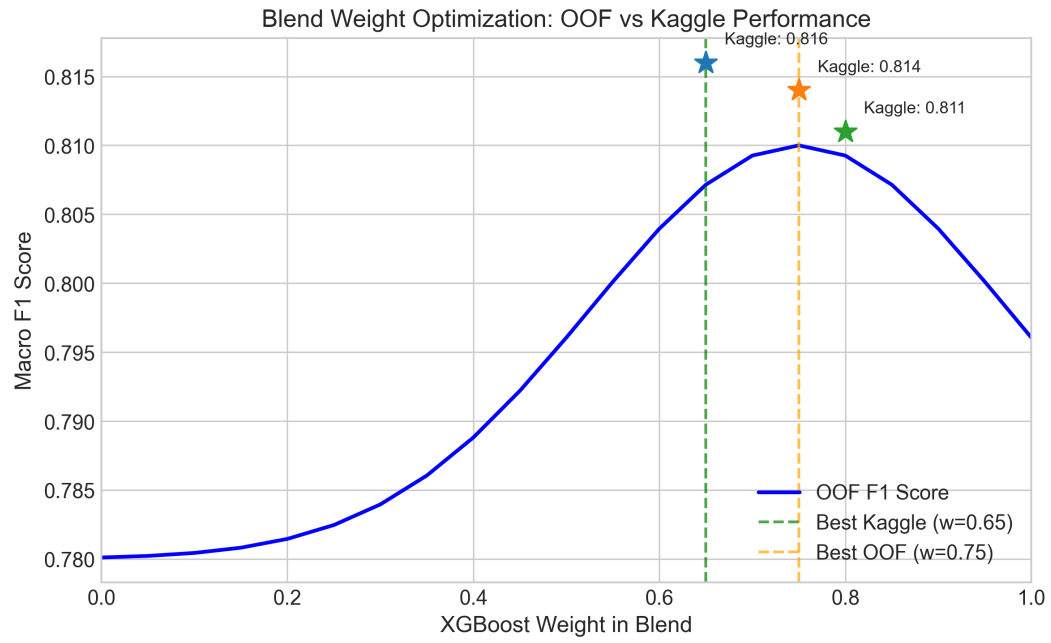


Figure 2: Performances for different XGB weights